

plt_sim_v2

JLT, SAA

2025-06-14

Adapted from plt_sim.R 2025-05-29

Simulation

Simulation parameters

Assign distributions to parameters for spawner behaviour. Also years and days for surveys.

```
log_run_mu          <- 10 # lognormal
log_run_sigma       <- 1
arrival_mu          <- 35 # normal
arrival_sigma       <- 5
arrival_spread_mu   <- 5 # lognormal
arrival_spread_sigma_log <- 0.3
exit_spread_mu      <- 5 # lognormal
exit_spread_sigma_log <- 0.3
exit_lag           <- 10 # lognormal
exit_lag_sigma_log  <- 0.3
years              <- 10
days_min          <- 1
days_max          <- 100
rnbinom_size       <- 10 # sampling error
```

Spawner residence

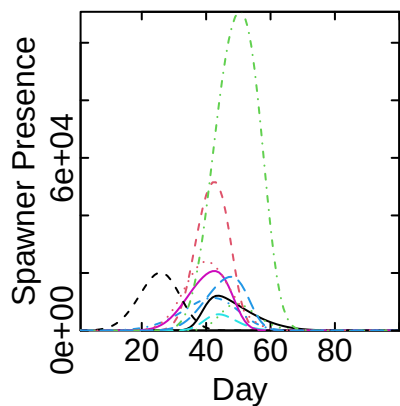
From cumulative arrived minus cumulative exited for each day.

```

# run size, spawner abund. both params logs
N_total <- round(rlnorm(years, log_run_mu, log_run_sigma))
# 95% 3.1k to 156k
arrival <- rnorm(years, arrival_mu, arrival_sigma)
arrival_spread <- rlnorm(years, log(arrival_spread_mu), arrival_spread_sigma_log)
# e.3 = 1.35, 15:50
exit_spread <- rlnorm(years, log(exit_spread_mu), exit_spread_sigma_log)
exit_lag <- rlnorm(years, log(exit_lag), exit_lag_sigma_log)
# range 5 to 15
output <- sapply(1:years, function(i){
  x <- days_min:days_max
  entered <- pnorm(x, arrival[i], arrival_spread[i])
  exited <- pnorm(x, arrival[i] + exit_lag[i], exit_spread[i])
  present <- entered * (1-exited) * N_total[i] # only present saved
})

par(tcl=0.2, mgp=c(1,0,0), pty='s', xaxs='i', yaxs='i')
matplot(output, type = 'l', xlab='Day', ylab='Spawner Presence')
axis(3, labels=F); axis(4, labels=F)

```



Surveys

Define sampling as a fraction randomly selected of all days. Add negative binomial sampling error. Remove simulated values of zero and sampled observations of zero.

```

full_surveyed.df <- data.frame(
  N = round(c(output)),
  sampled = rnbino(n = length(output), mu = output, size = rnbino_size),
  year = rep(1:years, each = 100),

```

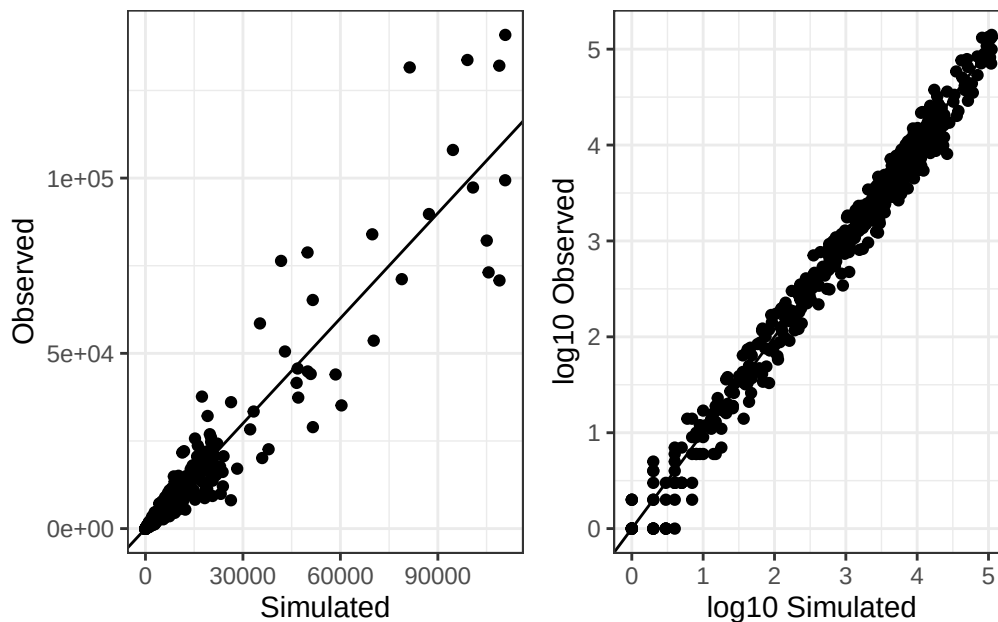
```

    day = 1:100)

sampled.df <- full_surveyed.df %>%
  group_by(year) %>%
  sample_frac(size = 0.9) # see slice_sample
  # delete true = 0
sampled.df2 <- sampled.df %>% filter( (sampled > 0) & (N > 0) )

p1 <- ggplot(sampled.df2, aes(N, sampled)) + theme_bw()+
  geom_point() +
  geom_abline(slope=1, intercept=0) +
  labs(x="Simulated",y="Observed")
p2 <- ggplot(sampled.df2, aes(log10(N), log10(sampled)) ) + theme_bw()+
  geom_point()+
  geom_abline(slope=1, intercept=0) +
  labs(x="log10 Simulated",y="log10 Observed")
p1+p2 # patchwork for plots layout

```

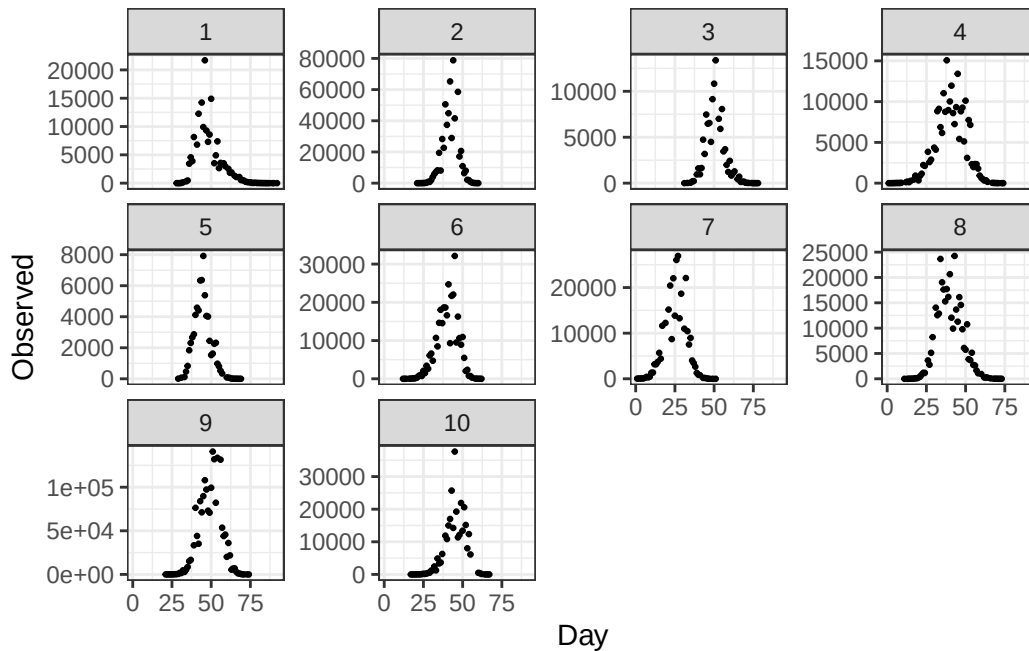


```

#ggsave('sampled_df.pdf',width=7, height=7)

ggplot(sampled.df2, aes(x = day, y = sampled)) + theme_bw() +
  geom_point(size=0.5) +
  labs(x="Day",y="Observed") +
  facet_wrap(~year, scales='free_y')

```



Fit

Priors for Stan

Affects sampling of the parameters space. Four parameters are treated as groups (multilevel) of annual values: *arrival_timing*, *arrival_spread*, *exit_lag*, *exit_spread*. The priors define the distributions for parameters defining distribution of annual values. Annual values are sampled (fitted) from the group distribution, and group parameters are simultaneously fitted, thereby “sharing information” to improve the fit for each annual value. Note *log_runs* is not a group, the same prior applies to each year.

In what follows, *exit_lag_sigma* uses the value for *spread_sigma*. The Stan code renames *count_dispersion* as *log(live_phi)* where *live_phi* is the size parameter for the negative binomial distribution.

```
priors <- data.frame(prior = c("log_runs_mu", "arrival_mu", "arrival_sigma", "spread_mu", "spread_sigma", "exit_lag_mu", "exit_lag_sigma", "count_dispersion"),
                    v1 = c(10, 40, 0, 5, 0, 10, 1),
                    v2 = c(1, 5, 10, 1, 5, 2, 0.2))
pr_tbl = cbind(priors,
               pdd=c('lnorm','norm','exp','norm','exp','norm','norm'))
kable(pr_tbl,digits=3)
# previously
```

Table 2

```

sp_dat = list(n_priors = nrow(priors),
              priors = data.matrix(priors[,-1]),
              n_years = max(sampled.df2$year),
              year = as.numeric(factor(sampled.df2$year)),
              day = sampled.df2$day,
              n_obs = nrow(sampled.df2),
              live_counts = sampled.df2$sampled,
              live_phi = 10,
              arrival_mu = arrival_mu)

```

```

# v1 = c(10, 40, 0, log(8-1), 0, log(11-1), 1),
# v2 = c(1, 5, 10, 0.2, 5, 0.3, 0.2))

```

Table 1: Priors for Stan sampling of parameters space.

prior	v1	v2	pdd
log_runs_mu	10	1.0	lnorm
arrival_mu	40	5.0	norm
arrival_sigma	0	10.0	exp
spread_mu	5	1.0	norm
spread_sigma	0	5.0	exp
exit_lag_mu	10	2.0	norm
count_dispersion	1	0.2	norm

Data list for Stan

The priors are part of the Stan data list. Convergence of the HMC sampling chains is improved by providing initial values for *arrival_mu*.

Stan sampling

```

mod <- stan_model("spawners_m8.stan")
z <- Sys.time()
fit <- sampling(mod, data = sp_dat,
               chains = 4, cores = 4,

```

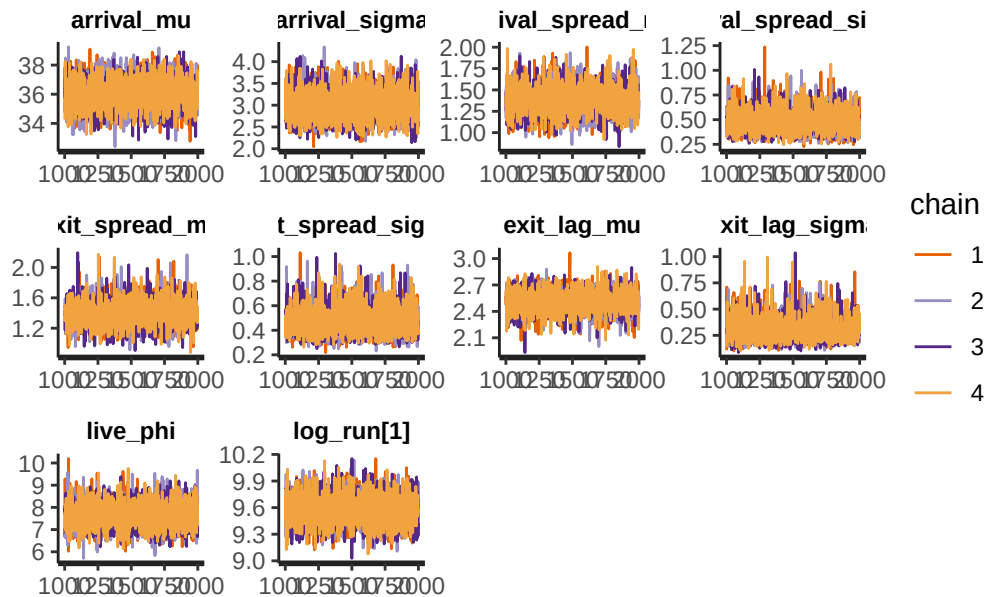
```

init = replicate(4, list(arrival_mu = 35), simplify = FALSE))
print(paste0('elapsed time:', format(Sys.time() - z) ))

traceplot(fit)

```

'pars' not specified. Showing first 10 parameters by default.



```
post <- rstan::extract(fit) # several versions of extract()
```

Fitted runs.

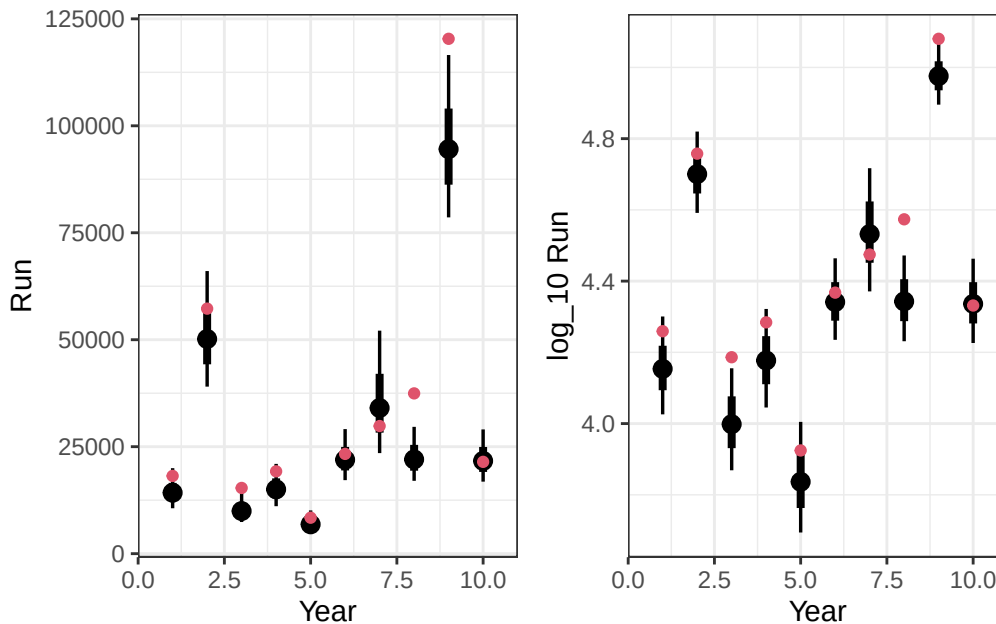
Spawner abundance estimates presented logged and arithmetic plots. Ratio $\log_{10} / \log_e = 0.4342945$

```

run_fit <- spread_draws(fit, log_run[year]) %>%
  filter(.chain %in% c(3:4))
p1 <- ggplot(run_fit, aes(x = year, y = exp(log_run))) + theme_bw() +
  stat_pointinterval() + labs(x="Year", y= "Run") +
  geom_point(data = data.frame(
    year = 1:years, log_run = log(N_total) ),
    color = 2)
p2 <- ggplot(run_fit, aes(x = year, y = 0.4343*log_run)) + theme_bw() +
  stat_pointinterval() + labs(x="Year", y= "log_10 Run") +

```

```
geom_point(data = data.frame(
  year = 1:years, log_run = log10(N_total)),
  aes(x=year, y=log_run), color = 2)
p1+p2
```



Fitted parameters

The annual values for spawner *arrival_timing*, *arrival_spread*, *exit_lag*, and *exit_spread* are presented as marginal posterior distributions (confidence limits) and compared to the simulated value. The goal is finding any consistent bias.

```
p1 <- spread_draws(fit, arrival[year]) %>%
  filter(.chain %in% c(3:4)) %>%
  ggplot(aes(x = year, y = arrival)) + theme_bw() +
  stat_pointinterval()+ labs(x="Year", y="Arrival Timing") +
  geom_point(data = data.frame(year = 1:years, arrival = arrival), color = 2)

p2 <- spread_draws(fit, arrival_spread[year]) %>%
  filter(.chain %in% c(3:4)) %>%
  ggplot(aes(x = year, y = arrival_spread)) + theme_bw() +
  stat_pointinterval()+ labs(x="Year", y="Arrival Spread") +
  geom_point(data = data.frame(year = 1:years, arrival_spread = arrival_spread), color = 2)
```

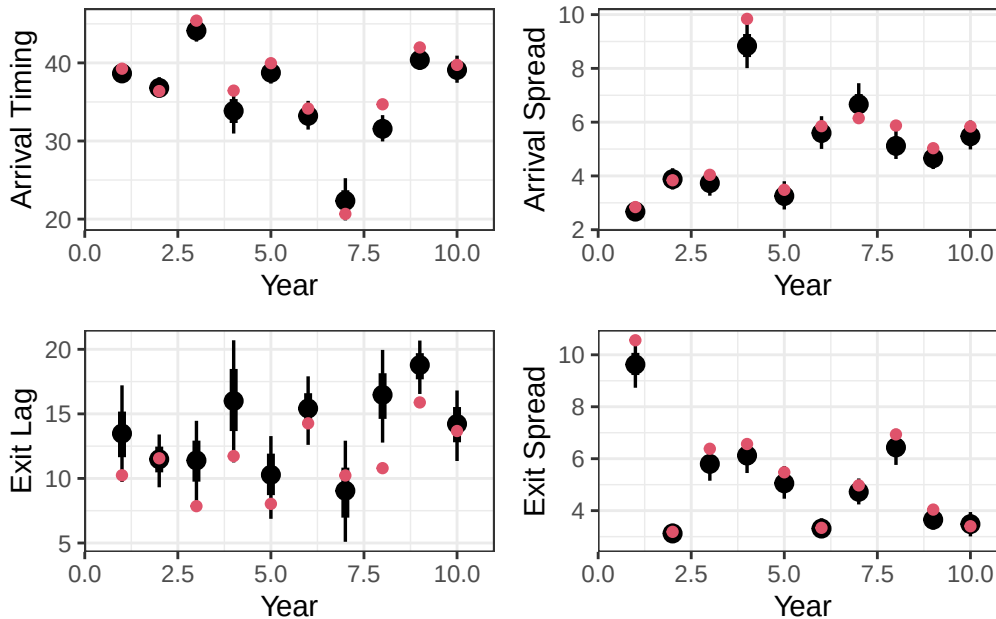
```

p3 <- spread_draws(fit, exit_lag[year]) %>%
  filter(.chain %in% c(3:4)) %>%
  ggplot(aes(x = year, y = exit_lag)) + theme_bw() +
  stat_pointinterval()+ labs(x="Year", y="Exit Lag") +
  geom_point(data = data.frame(year = 1:years, exit_lag = exit_lag), color = 2)

p4 <- spread_draws(fit, exit_spread[year]) %>%
  filter(.chain %in% c(3:4)) %>%
  ggplot(aes(x = year, y = exit_spread)) + theme_bw() +
  stat_pointinterval()+ labs(x="Year", y="Exit Spread") +
  geom_point(data = data.frame(year = 1:years, exit_spread = exit_spread), color = 2)

p1+p2+p3+p4

```



Spawner presence

The pattern of spawner distribution is recreated from the fitted parameters and presented with 95% confidence limits. This is compared to the observations.

```

spawn_curves.df <- data.frame()
for(j in 1:ncol(post$log_run)){
  live <- sapply(1:100, FUN = function(x) {
    entered <- pnorm(x, post$arrival[,j], post$arrival_spread[,j])

```



```

    exited <- pnorm(x, post$arrival[,j] + post$exit_lag[j],
                    post$exit_spread[,j])
    exp(post$log_run[,j] + log(entered - (entered * exited)))
  })

spawn_curves.df <- bind_rows(spawn_curves.df,
  data.frame(year = j,
    day = 1:100,
    fish = apply(live, 2, median),
    l95 = apply(live, 2, quantile, probs = 0.025),
    u95 = apply(live, 2, quantile, probs = 0.975)))
}

ggplot(spawn_curves.df, aes(x = day, y = fish)) + theme_bw() +
  geom_line()+
  geom_ribbon(aes(ymin = l95, ymax = u95), color = NA, alpha = 0.2)+
  facet_wrap(~year, scales = "free_y") +
  geom_point(data = sampled.df2, aes(x = day, y = sampled), size = 0.8) +
  scale_y_continuous(
    labels = label_number(scale = 1e-3, suffix = "k"),
    name = "Spawners present (thousands)", expand = expansion(mult = c(0, 0.02))) +
  theme(strip.background = element_rect(color="NA", fill="NA"))+
  scale_x_continuous(
    labels = ~ format(as.Date(.x, origin = "2023-12-31"), "%b"),
    breaks = as.numeric(as.Date(c("2024-09-01", "2024-10-01", "2024-11-01")) - as.Date("2023-12-31"))
  ) +
  theme(panel.grid.minor = element_blank())

ggsave('spawn_curves.pdf',width=14, height=14)

```

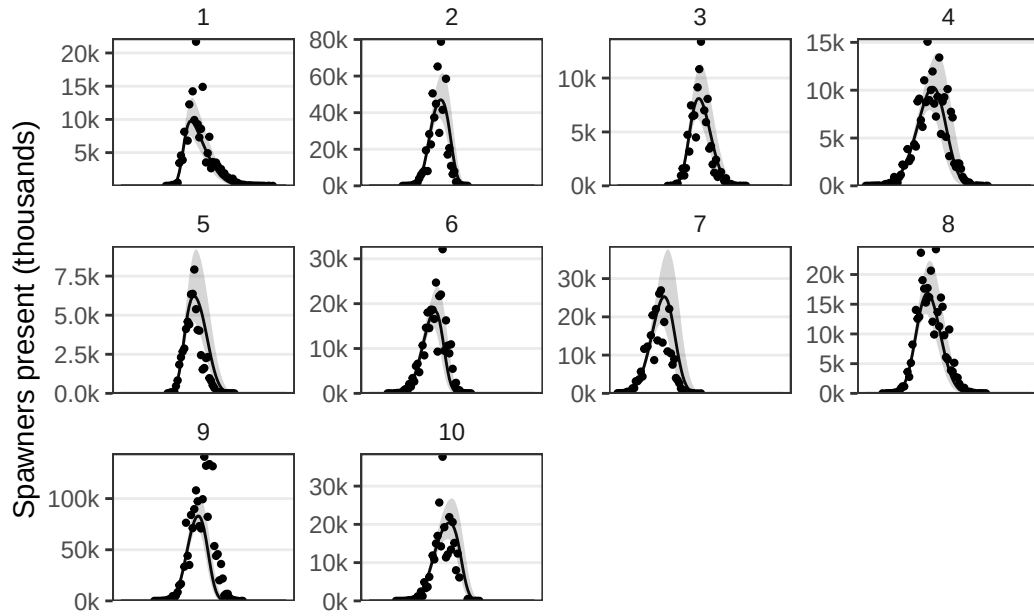


Figure 1: Estimated spawner presence, with 95% CL, from fitted parameters. Observed values are points.